Class 1: Code & Programming Foundations

From Zero to AI Agents — Seven-Class Edition

Why Learn Code in the Age of AI?

What AI Tools Actually Do

Claude Code, Codex, Copilot, Cursor — they all do one thing: **Generate code.**

The magic is code. The output is code. The thing that runs on servers, powers apps, and costs your company money is **code**. Your job is still to **steer it**. If you can read the shape of that code, you can ask for better work, catch bad work, and make decisions that are not vibes.

It’s Code All the Way Down

Even “no-code” tools generate code under the hood.

- Zapier writes API calls
- Vercel runs build scripts
- AI agents execute shell commands
- Websites are HTML, CSS, and JavaScript

You may never write code yourself. But code is what’s running — and someone needs to understand it.

Why This Matters for You

In your next job, you will:

- Review AI-generated code in pull requests
- Evaluate technical vendors and their architectures
- Make build-vs-buy decisions that cost real money
- Debug production issues when things break
- Manage engineers who speak in code

Without the basics, you'll be **flying blind**.

The Goal of This Class

The goal is **not** to make you a software developer.
But you **will** be prototyping with AI. You'll vibe-code internal tools, A/B tests, dashboards. AI makes that easy now.
What AI won't tell you:

- Your prototype is **leaking user data** or **exposing API keys**
- You're running experiments against **production data** instead of a safe copy
- Your company already has **code and APIs you could reuse** instead of building from scratch
- Your database connection is **open to the internet** with no authentication

> **Important**
>
> Don't be the employee who shipped a vibecoded A/B test to real users and leaked the company's API keys. **Understanding the basics is what keeps you safe, fast, and effective.**

What Success Looks Like Today

The goal is not “become a software engineer overnight.” By the end of this class you should be able to:

- Read a small program without panic.
- Explain what it takes in, what it changes, and what it returns.
- Change one behavior deliberately.
- Test whether the change actually worked.
- Run a website on your own machine — and explain why nobody else can see it.

Today’s mental model: a program is a precise recipe that transforms inputs into outputs. Everything else — websites, models, agents, databases — is an elaboration of that recipe.

Today’s Agenda

1. **What is code?** — recipes, binary, and how code actually runs
2. **The building blocks** — variables, conditionals, loops, functions (live, in your browser)
3. **Errors are information** — syntax, runtime, and the dangerous kind
4. **Programming languages** — why so many, and why Python
5. **The terminal** — texting your computer
6. **Clean code** — readability, WTF/minute, and how real projects are organized
7. **Git & GitHub** — time travel for code
8. **Localhost** — running a website on your own machine
9. **Your tools + AI coding agents** — Claude Code, Codex, YOLO mode
10. **Exercise** — a business-rule engine you can see in a browser

What is Code?

Code is a Recipe

Imagine you're writing a recipe. You list ingredients, describe each step, and someone follows your instructions to make a dish.

Code is exactly that — a set of instructions that tells a computer what to do.

The difference? Computers are **incredibly literal**.

If your recipe says "add salt" but doesn't say how much, a human figures it out. A computer would crash (or add infinite salt).

Code is a Contract with a Machine

Humans tolerate ambiguity. Machines do not.
“Give loyal customers a discount” is a sentence. To become code, it must become a decision with exact inputs, thresholds, and exceptions:

- What counts as “loyal”? Two years? Ten purchases?
- How much discount? On everything?
- What if the customer is loyal *and* has an unpaid invoice?

> **Tip**
>
> This translation — from fuzzy policy to exact rules — is most of what programming *is*. The typing is the easy part.

How Computers Think

At the lowest level, computers only understand **binary** — ones and zeros.

```
01001000 01100101 01101100 01101100 01101111 = "Hello"
```

Every photo, video, song, and app on your phone is ultimately a sequence of `1`s and `0`s.
Writing programs in binary would be insane. That’s why we have **programming languages** — human-readable ways to write instructions that get translated into binary.

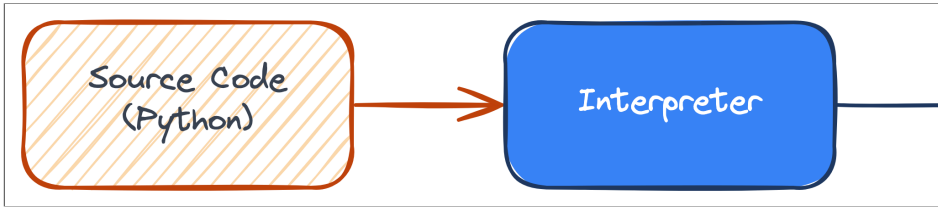
How Code Runs

The journey from your code to computer action:

1. **You write source code** in a language like Python
2. An **interpreter** (or compiler) translates it to machine instructions
3. The **CPU executes** those instructions

> **Tip**
>
> **Compiled languages** (C, Go, Rust) translate all the code at once before running. Like translating a whole book.
>
> **Interpreted languages** (Python, JavaScript) translate line by line as they run. Like having a live translator.

How Code Runs — Visually



Source Code is Not the Product

Source code is a blueprint. The running program, its data, its configuration, and the environment around it are the product. That distinction explains why “it worked on my laptop” is a real failure mode — the code was the same, but everything around it was different.

> **Note**
>
> Abstraction is what lets people build larger things. You write `print()`, not instructions for a screen. You call a library, not a network chip. Each layer hides detail — and introduces a dependency you should understand well enough to inspect.

Your First Program

Python Code | Start Over | Run Code

```python
# Your first program — the classic "Hello, World!"
print("Hello, World!")

# Variables: named boxes that hold values
name = "Alice"
age = 25
is_student = True

print(f"My name is {name}, I'm {age} years old")
print(f"Student? {is_student}")
```

```
_pyodide_editor_1 = ▸Object {code: null, options: Object, indicator: Ge}
```

Every Program: Input → Processing → Output

Strip away the frameworks and every program has the same shape:

Python Code Start Over Run Code

```
revenue = 120_000          # input
discount_rate = 0.10       # input
discount = revenue * discount_rate   # processing
print(discount)            # output
```

_pyodide_editor_2 = ▸Object {code: null, options: Object, indicator: Ge}

Input: revenue and rate. Processing: multiplication. Output: the discount.

Instagram? Input: your scroll. Processing: a ranking algorithm. Output: the next photo. Same shape, more zeros in the budget.

Pseudocode: Think Before You Type

Professionals sketch the logic in plain language first. This is called **pseudocode**:

```
for each customer:
    if customer is enterprise AND has no overdue invoices:
        discount = 15%
    otherwise:
        discount = 5%
    record the discount
```

No syntax, no language, no compiler — just the *decision*, written down where a human can argue with it.

> **Tip**
>
> Pseudocode is also the single best way to brief an AI coding agent. If you can write the pseudocode, the agent can write the code — and you can check its work against your own logic.

The Building Blocks of Every Program

Building Blocks: Variables

Every program is built from just a few pieces. Let's start with **variables** — storing data.

Python Code Start Over Run Code

```
# Numbers
price = 19.99
quantity = 3
total = price * quantity
print(f"Total: ${total}")

# Text (strings)
greeting = "Hello"
target = "World"
message = greeting + " " + target + "!"
print(message)
```

_pyodide_editor_3 = ▸Object {code: null, options: Object, indicator: Ge}

State is What the Program Remembers

Variables give names to values while a program runs. This is the program's **state** — its working memory.

Python Code | Start Over | Run Code

```
customer_name = "Maya"
is_enterprise = True
open_invoices = 3

print(f"{customer_name} — enterprise: {is_enterprise}, oper
```

```
_pyodide_editor_4 = ▸Object {code: null, options: Object, indicator: Ge}
```

When something goes wrong in a program, the first question is almost always: *what was the state at that moment?*

Types Are Categories with Rules

Every value has a **type**, and types come with rules about what you can do with them:

Type	Example	Useful for
Number	`42`, `3.14`	money, counts, scores
String	`"Maya"`	names, IDs, messages
Boolean	`True`	yes/no decisions
List	`["A", "B"]`	ordered collections
Dictionary	`{"tier": "gold"}`	labeled facts

Types Prevent Nonsense

`"100" + "20"` produces text. `100 + 20` produces arithmetic.

Python Code | Start Over | Run Code

```
print("100" + "20")   # strings glue together
print(100 + 20)       # numbers add up

# This is a classic real-world bug: numbers stored as text
price_from_spreadsheet = "19.99"
print(type(price_from_spreadsheet))
print(float(price_from_spreadsheet) * 3)   # convert first,
```

_pyodide_editor_5 = ▸Object {code: null, options: Object, indicator: Ge}

The computer is not being pedantic; it is asking what you mean. Half of all data-cleaning work is fixing numbers that someone stored as text.

Strings Deserve a Minute

You'll spend more time manipulating text than doing math. Python's **f-strings** are the workhorse:

Python Code | Start Over | Run Code

```
name = "Maya"
revenue = 120000
tier = "gold"

# f-strings: drop variables straight into text
print(f"{name} ({tier}) — annual revenue ${revenue:,}")

# Formatting controls: commas, decimals, percentages
rate = 0.1275
print(f"Effective rate: {rate:.1%}")

# Useful string methods
email = "  Maya.Chen@Example.COM  "
print(email.strip().lower())          # clean it up
print("gold" in tier)                 # membership check
print("2024-Q3-report.pdf".split("-")) # break apart
```

_pyodide_editor_6 = ▸Object {code: null, options: Object, indicator: Ge}

Note

> Every messy spreadsheet export, every log file, every API response is text that needs cleaning. These five methods (`strip`, `lower`, `split`, `in`, f-strings) handle 80% of it.

Building Blocks: Conditionals

Conditionals let your program make decisions.

Python Code Start Over Run Code

```
1 temperature = 72
2
3 if temperature > 85:
4     print("It's hot! Stay hydrated.")
5 elif temperature > 60:
6     print("Nice weather!")
7 else:
8     print("It's cold! Wear a jacket.")
```

_pyodide_editor_7 = ▸Object {code: null, options: Object, indicator: Ge}

Good Conditions Name the Business Rule

Conditions turn policy into behavior. Compare:

Python Code | Start Over | Run Code

```
is_enterprise = True
open_invoices = 0

# Buried logic — works, but opaque:
if is_enterprise and open_invoices == 0:
    discount_rate = 0.15
else:
    discount_rate = 0.05

# Named logic — the rule is inspectable:
eligible_for_discount = is_enterprise and open_invoices ==
discount_rate = 0.15 if eligible_for_discount else 0.05
print(f"Eligible: {eligible_for_discount}, rate: {discount
```

```
_pyodide_editor_8 = ▸Object {code: null, options: Object, indicator: Ge}
```

`eligible_for_discount` is a business rule with a name. Anyone — your CFO, an auditor, an AI agent — can find it and question it.

Building Blocks: Loops

Loops repeat things so you don't have to.

Python Code | Start Over | Run Code

```
# Count from 1 to 5
for i in range(1, 6):
    print(f"Count: {i}")

# Loop through a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"I like {fruit}")
```

```
_pyodide_editor_9 = ▸Object {code: null, options: Object, indicator: Ge}
```

Loops in the Wild: Totaling Invoices

The most common loop in business software — walk through records, accumulate a result:

Python Code | Start Over | Run Code

```python
invoices = [1200, 800, 4200]
total = 0

for invoice in invoices:
    total = total + invoice

print(f"Total outstanding: ${total:,}")

# Python being Python, there's usually a shortcut:
print(f"Same thing: ${sum(invoices):,}")
```

_pyodide_editor_10 = ▸Object {code: null, options: Object, indicator: Ge}

Building Blocks: Functions

Functions are reusable blocks of logic.

Python Code | Start Over | Run Code

```python
def greet(name, enthusiasm=1):
    """Create a greeting with optional excitement"""
    exclamation = "!" * enthusiasm
    return f"Hello, {name}{exclamation}"

print(greet("Alice"))
print(greet("Bob", enthusiasm=3))
```

_pyodide_editor_11 = ▸Object {code: null, options: Object, indicator: Ge}

A Function is a Tiny Product Boundary

Functions package a decision:

Python Code | Start Over | Run Code

```python
def calculate_discount(revenue, tier):
    if tier == "gold":
        return revenue * 0.15
    return revenue * 0.05

print(calculate_discount(120_000, "gold"))
print(calculate_discount(120_000, "standard"))
```

_pyodide_editor_12 = ▸Object {code: null, options: Object, indicator: Ge}

A function has a name, inputs, an output, and a promise. That makes it reusable — and **testable**.

> **Tip**
>
> Good functions do one recognizable job. `calculate_discount()` is easier to test and change than `calculate_discount_and_send_email_and_update_dashboard()`.

Names Are a User Interface

`eligible_for_discount` explains intent. `x` forces everyone — including future you and an AI agent — to rediscover it.

Bad name	Good name
`x`	`monthly_revenue`
`flag`	`is_overdue`
`data2`	`filtered_customers`
`temp`	`discount_before_cap`

Comments explain why, not what. The code says *what* it does. A comment earns its place when it records a surprising constraint, policy choice, or tradeoff:

```python
# Legal requires 0% discount on overdue accounts (2023 audit finding)
if has_overdue_invoice and not approved_exception:
    return 0
```

Lists versus Dictionaries

Use a **list** when order matters. Use a **dictionary** when labels matter.

Python Code | Start Over | Run Code

```
# List: an ordered collection — "the queue of invoices to
invoices = [1200, 800, 4200]
print(f"First invoice: {invoices[0]}")

# Dictionary: labeled facts — "everything we know about th
customer = {"name": "Maya", "tier": "gold", "revenue": 120
print(f"{customer['name']} is a {customer['tier']} custome

# Real data is usually both: a list of dictionaries
customers = [
    {"name": "Maya", "tier": "gold"},
    {"name": "Ben", "tier": "standard"},
]
for c in customers:
    print(f"  {c['name']}: {c['tier']}")
```



```
_pyodide_editor_13 = ▸Object {code: null, options: Object, indicator: Ge}
```

> **Note**
>
> A “list of dictionaries” is the shape of almost everything: rows in a spreadsheet, records in a database, results from an API. Get comfortable with it.

Data Shape is a Management Decision

The easiest model or dashboard in the world cannot save data with unclear definitions, missing owners, or inconsistent IDs. Before anyone writes code, someone has to answer:

- What exactly is a “customer”? A person? A company? A billing account?
- Which field is the unique ID, and who guarantees it stays unique?
- What does an empty value *mean* — zero, unknown, or not applicable?

> **Important**
>
> These are not engineering questions. They’re **business questions that get frozen into code**. Answer them wrong and every downstream system inherits the confusion.

Putting It All Together: A Mini Report

All four building blocks in one small, real program — the kind an analyst might actually write:

Python Code | Start Over | Run Code

```
customers = [
    {"name": "Maya", "tier": "gold", "revenue": 120000, "c
    {"name": "Ben", "tier": "standard", "revenue": 45000,
    {"name": "Aisha", "tier": "strategic", "revenue": 3000
    {"name": "Leo", "tier": "gold", "revenue": 87000, "ove
]

def discount_rate(tier, overdue):                  # functi
    if overdue:                                    # condit
        return 0.0
    rates = {"standard": 0.05, "gold": 0.10, "strategic":
    return rates[tier]

total_discount = 0                                 # variab
for c in customers:                                # loop
    d = c["revenue"] * discount_rate(c["tier"], c["overdue
    total_discount += d
    print(f"{c['name']:<8} {c['tier']:<10} ${d:>10,.0f}")

print(f"\nTotal discounts this quarter: ${total_discount:,
```

```
_pyodide_editor_14 = ▸Object {code: null, options: Object, indicator: Ge}
```

Twenty lines. A pricing policy, applied to data, producing a report. **This is what most business software is.**

How Code Executes — All 4 Building Blocks

Key Takeaway

> **Important**
>
> All software — from Instagram to self-driving cars — is built from these same building blocks: **variables, conditionals, loops, and functions**.
>
> The complexity comes from combining them in clever ways.

Errors Are Information

Three Flavors of Broken

Syntax errors — the code cannot be read. Usually a missing quote, bracket, colon, or indentation. The program has not started yet.
Runtime errors — the code started and hit a bad situation. A missing dictionary key, dividing by zero, asking for a file that does not exist.
Logic errors — the dangerous kind. The program runs and returns a believable wrong answer. No error message, no crash — just a discount that's 10x too big, quietly, on every order.

> **Important**
>
> Syntax and runtime errors announce themselves. Logic errors hide. **These are why testing matters.**

Try It: Read a Traceback

Run this — it crashes on purpose. Read the error message from the **bottom up**:

Python Code | Start Over | Run Code

```
1 customer = {"name": "Maya", "tier": "gold"}
2
3 print(customer["name"])
4 print(customer["revenue"])   # KeyError: we never stored "r
```

```
_pyodide_editor_15 = ▸Object {code: null, options: Object, indicator: Ge}
```

The last line names the failure (`KeyError: 'revenue'`). The lines above show *where* it happened. Now fix it — replace the failing line with `customer.get("revenue", 0)` and re-run.

> **Tip**
>
> Stack traces are maps, not insults. Read from the bottom: *what* failed? Then move upward: which line in *your* code supplied the bad input?

Try It: Catch the Logic Bug

This program runs without errors — and is **wrong**. Find the bug before you read the hint:

Python Code | Start Over | Run Code

```
def average_deal_size(deals):
    total = 0
    for deal in deals:
        total += deal
    return total / len(deals)

q3_deals = [50000, 72000, 0, 0, 61000]   # two deals fell t
print(f"Average deal size: ${average_deal_size(q3_deals):,.
```

_pyodide_editor_16 = ▸Object {code: null, options: Object, indicator: Ge}

The output says **$36,600**. The real average of *closed* deals is **$61,000**. Those zeros aren't deals — they're failures being averaged in, silently dragging the number down 40%.

> **Important**
>
> No error. No crash. A confident, wrong number heading straight into a board deck. Fix it: filter out the zeros (`deals = [d for d in deals if d > 0]`) — then ask the better question: *should* cancelled deals be zeros in this data at all?

The First Debugging Question

When a program is wrong, ask: **what value did I think this was, and what value is it really?**

Python Code | Start Over | Run Code

```
# A believable logic bug: revenue arrived as text
revenue = "120000"
discount = 0.10

# print(revenue * discount)   # <- uncomment: TypeError!

# Make the invisible visible:
print(f"revenue is {revenue!r}, type: {type(revenue).__nam

# Fix: convert, then compute
print(float(revenue) * discount)
```

_pyodide_editor_17 = ▸Object {code: null, options: Object, indicator: Ge}

Print the value. Print its type. **Make the invisible visible.**

The Error Decoder Ring

The Python errors you'll actually meet, and what they're really saying:

Error	Translation	Usual cause
`SyntaxError`	"I can't even read this"	missing `:`, quote, or bracket
`IndentationError`	"Your spacing is inconsistent"	mixed tabs/spaces, wrong nesting
`NameError`	"Never heard of that variable"	typo, or used before defined
`TypeError`	"You can't do *that* to *this*"	text + number, wrong argument
`KeyError`	"That dictionary has no such label"	missing field in your data
`IndexError`	"The list isn't that long"	asking for item 5 of 3
`ZeroDivisionError`	"You divided by zero"	empty data → average of nothing

> **Tip**
>

> Paste any error into Claude and it will explain it. But after a few weeks, you won't need to — this table covers 90% of real life.

Debugging is Hypothesis Testing

Debugging is the scientific method with worse lighting:

1. Reproduce the behavior.
2. Find the smallest surprising value or branch.
3. State a hypothesis.
4. Change one thing.
5. Re-run the same case.

> **Note**
>
> Change one thing at a time. If you change three things and it works, you've learned nothing — and probably added a new bug.

Tests Turn Examples into Executable Promises

If a policy says strategic customers get 15%, write the example once as a test and let the computer guard it on every change.

Python Code | Start Over | Run Code

```python
def calculate_discount(revenue, tier):
    if tier == "strategic":
        return revenue * 0.15
    return revenue * 0.05

# The test — an example that runs forever:
assert calculate_discount(100_000, "strategic") == 15_000
assert calculate_discount(100_000, "standard") == 5_000
print("All promises kept ✓")
```

_pyodide_editor_18 = ▸Object {code: null, options: Object, indicator: Ge}

Error handling is part of the experience. Good software tells a user what happened, preserves their work when possible, and gives the operator enough context to fix the underlying issue. "Something went wrong :(" is a design failure, not an engineering inevitability.

Bugs That Cost Real Money

Logic errors aren't academic. A few famous ones:

Year	Bug	Cost
1999	**Mars Climate Orbiter** — one team used metric units, another used imperial	$327M spacecraft lost
2012	**Knight Capital** — old trading code accidentally reactivated by a bad deploy	$440M in 45 minutes
2018–19	**Boeing 737 MAX (MCAS)** — software trusted a single faulty sensor	346 lives, ~$20B+
2024	**CrowdStrike update** — one malformed config file	~8.5M Windows machines down, airlines grounded

Important

Notice what these have in common: the code **ran fine**. No syntax errors, no crashes at ship time. The *assumptions* were wrong — and no test caught them.

When your team asks for time to "write tests" or "add review steps," this table is why.

Programming Languages

Why So Many Languages?

If code is a recipe, then programming languages are the different human languages you can write that recipe in.
A chocolate cake recipe in English, French, or Japanese — same result, different words.
Why so many? Because they optimize different things: speed, safety, approachability, browser reach, data work, and ecosystem.

Languages Are Tools

Different languages are optimized for different tasks:

Language	Best For	Example Use
Python	Data science, AI, scripting	ChatGPT, Instagram backend
JavaScript	Web browsers, frontend	Every website you visit
Go	Fast servers, cloud tools	Docker, Kubernetes
Java	Enterprise, Android apps	Banking systems
C/C++	Operating systems, games	Windows, Unreal Engine
Rust	Safe, fast systems	Firefox, Cloudflare

Why Python?

> **Tip**
>
> Python reads almost like English, has a massive ecosystem for AI/ML, and is the most popular language in data science. That's why we use it throughout this course.

Compiled languages (C, Go, Rust) — translate the entire program to machine code *before* running. Faster execution.
Interpreted languages (Python, JavaScript, Ruby) — translate code line-by-line *while* running. Slower, but you can run code instantly.
In truth it's a spectrum — many languages do both. The practical question: **when do you learn that the program is wrong** — before release, at startup, or under real user traffic?

The Supporting Cast

Two languages you'll meet constantly, even if you never "learn" them:

JavaScript and TypeScript — the web's language. JavaScript runs in every browser on Earth. TypeScript adds checks that catch mistakes before users do. (Much more in Class 2.)

SQL — the language of questions over data.

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

SQL isn't a general-purpose language — it does one thing (ask precise questions of structured data) and has done it since the 1970s. Learning SQL is one of the **highest-ROI skills** in tech. We'll run it live in Class 2.

FizzBuzz — The Classic Coding Exercise

The most famous programming interview question:

- Print numbers 1 to 20
- If divisible by 3, print "Fizz" instead
- If divisible by 5, print "Buzz" instead
- If divisible by both, print "FizzBuzz"

Python Code | Start Over | Run Code

```
for n in range(1, 21):
    if n % 3 == 0 and n % 5 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

_pyodide_editor_19 = ▸Object {code: null, options: Object, indicator: Ge}

The Same Idea, Different Languages

Python

Python Code | Start Over | Run Code

```
1 print("Hello, World!")
```

```
_pyodide_editor_20 = ▸Object {code:
null, options: Object, indicator: Ge}
```

JavaScript

```
console.log("Hello, World!");
```

Go

```
package main
import "fmt"
func main() {
    fmt.Println("Hello, World!")
}
```

Java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Python wins on simplicity — **one line** vs four or more.

Why AI Agents Love Python & JavaScript

Not all languages get equal AI support. Models write the best code in the languages with the most public training data:

- **Python and JavaScript dominate GitHub** — so models have seen millions of examples of every pattern
- Popular frameworks (React, FastAPI, Next.js) are heavily represented — agents scaffold them almost flawlessly
- Niche languages and internal frameworks get noticeably worse output

> **Note**
>
> Practical consequence: when prototyping with AI, **choosing a mainstream stack is a productivity decision**, not just a technical one. The agent is dramatically better at the boring, popular choice.

One More “Language”: Markdown

Markdown is how technical people write documents — READMEs, wikis, pull request descriptions, and (increasingly) prompts for AI agents:

```
# Project Title
## Setup
1. Install dependencies with `npm install`
2. Copy `.env.example` to `.env`

**Warning:** never commit the `.env` file.
```

for headings, **bold**, `code`, – for lists. That’s basically it.

> **Note**
>
> These slides are written in Markdown. Your AI agent’s instructions will be Markdown files. The README in every GitHub repo is Markdown. Ten minutes of learning, used daily forever.

Libraries Are Borrowed Capability

You rarely build encryption, charting, HTTP requests, or model APIs from scratch.
You combine libraries — and inherit their assumptions, licenses, and security risks.

```
import requests        # someone else's HTTP code
import pandas as pd    # someone else's data tables
from openai import OpenAI   # someone else's AI client
```

Three lines, three teams of engineers working for you for free.

> **Warning**
>
> Free code is not risk-free code. In 2024, a single volunteer-maintained library (xz) nearly shipped a backdoor into most of the world’s Linux servers. Know what you depend on.

Versions Make Collaboration Reproducible

Two people with different package versions can run the same source code and get different behavior. Lockfiles and environment definitions make the dependency set explicit. **Dependencies are part of the product.** “It works” includes the exact packages and versions required to make it work somewhere else.

> **Tip**
>
> **Reproducibility is a leadership skill.** If a result cannot be rerun, inspected, and explained by another person, it is much harder to trust in a product, model, or boardroom decision.

The Terminal

What Is the Terminal?

The terminal is just a **text-based way to talk to your computer**. Instead of clicking buttons and dragging icons, you type commands.
Your computer's files form a **tree** — folders inside folders, starting from a root. The Finder/Explorer shows you the tree with icons; the terminal shows you the same tree with text:

```
/Users/you/
  ├── Documents/
  ├── Desktop/
  └── projects/
      └── networking-tracker/
          ├── app/
          └── README.md
```

At any moment you are standing *somewhere* in this tree — that's your **working directory**.
Think of it like **texting your computer** instead of pointing at things. Same computer, same capabilities — just a different interface.

> **Tip**
>
> Every developer, every server, and every AI tool ultimately runs through a terminal. Learning it is like learning to drive stick — not strictly necessary, but it unlocks everything.

How to Open It

Mac

1. Press **Cmd + Space** (Spotlight)
2. Type **"Terminal"**
3. Press Enter

Or find it in Applications → Utilities → Terminal

Windows

1. Press **Win** key
2. Type **"PowerShell"**
3. Press Enter

Or install **Windows Terminal** from the Microsoft Store for a better experience.

You Are Now a Hacker

Congratulations. You opened a terminal.
You are now officially a hacker.

Basic Terminal Commands

Just enough to get around:

Mac / Linux

```
pwd             # Where am I?
ls              # What's in this folder?
cd Documents    # Go into Documents
cd ..           # Go back up one folder
mkdir my-app    # Create a new folder
```

Windows (PowerShell)

```
pwd             # Where am I?
dir             # What's in this folder?
cd Documents    # Go into Documents
cd ..           # Go back up one folder
mkdir my-app    # Create a new folder
```

> **Note**
>
> **To stop a running command: Ctrl + C** (both Mac and Windows). This is your emergency brake — use it whenever something is stuck or you want to cancel.
>
> That's it. That's all you need to know to get started. Everything else you can look up — or ask Claude.

Anatomy of a Command

Every terminal command has the same grammar:

```
git   commit   -m   "Fix pricing bug"
 │       │     │          │
 │       │     │          └── argument (the data)
 │       │     └── flag (an option, starts with - or --)
 │       └── subcommand (which action)
 └── program (which tool)
```

Once you see this pattern, unfamiliar commands stop being scary:

```
npm install -g vercel        # program: npm, action: install, flag: -g (global), target: vercel
python app.py --debug        # program: python, argument: app.py, flag: --debug
claude --dangerously-skip-permissions   # you'll meet this one shortly
```

> **Tip**
>
> Don’t memorize commands — learn to *read* them. When an AI agent proposes running something in your terminal, this grammar is how you sanity-check it before saying yes.

Files Are the Real Interface

Code agents, deployments, test runners, and Git all coordinate through **files**.
The terminal is how you see the same reality your tools see: which folder you’re in, which files exist, what changed.

> **Important**
>
> Know where the project is and what changed **before** you ask an AI to change it. An agent pointed at the wrong folder will happily “fix” the wrong project.

Clean Code & Code Organization

Code Is Read Far More Than It Is Written

You write a line of code once. Then you read it — and your teammates read it, and every AI agent you point at the project reads it — dozens of times.

- The person most confused by your code will be **you, in three months**
- A new hire’s first month is almost entirely reading
- An AI agent has to read the surrounding code before it can change one line of it

> **Important**
>
> Readability is not decoration. It is the **throughput** of everyone who touches the code after you — including the machine you’re paying to help.

The WTF-per-Minute Metric

The only honest measure of code quality, from a famous Thom Holwerda cartoon: count how many times a reviewer says *“WTF?”* per minute.

Good code review

> “OK.”
> “Mm-hm.”
> “Sure.”
> “**WTF?**”
> “Right, ship it.”

Bad code review

> “**WTF?**”
> “**WTF?**”
> “**WTF** is `t2`?”
> “**WTF?**”
> “**WTF?**”

> **Tip**
>
> It’s a **social** metric, and that’s the point. It measures the gap between what the code does and what a reader expected it to

> do. You cannot measure that with a linter — only with a second human.

Spaghetti Code

Same discount logic you already know. Count the WTFs out loud:

```
def proc(c, r, t2, o, a=False):
    d = 0
    if o == True:
        if a == False:
            d = 0
        else:
            if t2 == "strategic":
                d = r * 0.15
            else:
                if t2 == "gold":
                    d = r * 0.10
                else:
                    d = r * 0.05
    else:
        if t2 == "strategic":
            d = r * 0.15
        elif t2 == "gold":
            d = r * 0.10
        else:
            d = r * 0.05
    # if r > 500000: d = d * 1.2   # enterprise bump? ask Dana
    print("Customer " + c + " gets " + str(d))
    return d
```

It **works**. That is what makes it dangerous.

What Made That Hard

Every smell forces the reader to stop and reconstruct something the author already knew:

Smell	The question you're forced to ask
`d`, `r`, `t2`, `o`, `a`	"Is `d` discount or days? Is `o` overdue or order?"
`0.15`, `0.10`, `0.05`	"Where do these come from? Who approved them?"
Nesting four levels deep	"Which condition am I inside right now?"
The tier rules, written twice	"Are these two copies still identical?"
`# enterprise bump? ask Dana`	"Is this still policy? Who is Dana?"
`proc()`	"Process *what*?"
Prints **and** returns	"Is this a calculator or a report? Can I reuse it?"

> **Note**

> None of these are bugs. Every one of them is a **future** bug, because the next person to change this code will misunderstand it.

The Same Logic, Cleaned Up

Identical behavior. Run it:

Python Code | Start Over | Run Code

```
1  DISCOUNT_RATES = {"strategic": 0.15, "gold": 0.10, "standa
2
3  def calculate_discount(revenue, tier, has_overdue_invoice,
4      """Return the discount owed, in dollars. Overdue accou
5      unless Finance has approved an exception (2023 audit f
6      if has_overdue_invoice and not approved_exception:
7          return 0
8      return revenue * DISCOUNT_RATES[tier]
9
10 print(calculate_discount(120_000, "gold", False))
11 print(calculate_discount(120_000, "strategic", True))
12 print(calculate_discount(120_000, "strategic", True, appro
```

```
_pyodide_editor_21 = ▸Object {code: null, options: Object, indicator: Ge}
```

Twenty-four lines became five. Nothing clever happened — the names just started telling the truth.

Guard Clauses Beat Nesting

Handle the exceptional case first, then leave. The rest of the function is the normal path:

Nested — read downward, hold state

```
if o == True:
    if a == False:
        d = 0
    else:
        d = rate(t2) * r
else:
    d = rate(t2) * r
```

Guard clause — leave early

```
if has_overdue_invoice and not approved_exception:
    return 0

return revenue * DISCOUNT_RATES[tier]
```

> **Tip**
>
> Deep nesting means the reader has to keep a stack of conditions in their head. Early returns pop that stack. **Indentation is a cost** — spend it deliberately.

Magic Numbers Have Names

A bare number in the middle of a function is a policy decision in disguise — and nothing on the line tells you *which* policy.

Magic

```
if days_overdue > 90:
    discount = 0
elif revenue > 500000:
    discount = revenue * 0.15 * 1.2
else:
    discount = min(revenue * 0.15, 25000)
```

Five numbers, five unrelated jobs. Which one is a rate? A cap? A deadline? A bump nobody remembers approving?

Named

```
COLLECTIONS_CUTOFF_DAYS = 90
ENTERPRISE_REVENUE = 500_000
STRATEGIC_RATE = 0.15
ENTERPRISE_BUMP = 1.2
MAX_DISCOUNT = 25_000

if days_overdue > COLLECTIONS_CUTOFF_DAYS:
    discount = 0
elif revenue > ENTERPRISE_REVENUE:
    discount = revenue * STRATEGIC_RATE * ENTERPRISE
else:
    discount = min(revenue * STRATEGIC_RATE, MAX_DIS
```

Same five numbers. Each one now says what it is — and has exactly one place to change.

> **Important**

> Notice `0.15` appears **twice** on the left, and you had to read carefully to spot it. That is how a rate change ships to half your customers and not the other half.

One Function, One Job

Remember this joke from earlier?

```
calculate_discount_and_send_email_and_update_dashboard()
```

The ands in the name are the code telling on itself. Split it:

```
discount = calculate_discount(revenue, tier, has_overdue_invoice)
send_discount_email(customer, discount)
update_dashboard(customer, discount)
```

Now you can test the math **without sending anyone an email**, reuse the calculation in a report, and change the dashboard without touching pricing policy.

> **Tip**
>
> If you cannot name a function without using “and,” it is doing more than one job.

Comments Explain Why, Not What

Worthless — restates the code

```
# add 1 to the counter
counter = counter + 1

# loop over customers
for c in customers:
    ...
```

The code already said that. Now there are two things to keep in sync.

Valuable — records a decision

```
# Legal requires 0% discount on overdue
# accounts (2023 audit finding)
if has_overdue_invoice and not approved_exception:
    return 0
```

Nothing in the code could have told you this.

> **Note**
>
> A comment earns its place when it records a surprising constraint, a policy choice, or a tradeoff. **Good names remove the need for "what" comments entirely.**

When One File Stops Working

Every project starts as a single `main.py`. That is correct — until it isn't.

- You scroll for ten seconds to find the function you want
- Two teammates change the file the same afternoon and collide on every commit
- You cannot tell what is safe to delete
- An AI agent has to read all 800 lines to change one pricing rule — slower, more expensive, and more likely to break something unrelated

> **Important**
>
> The symptom is never "the file is too long." The symptom is that **changing one thing means understanding everything.**

A Project Is a Folder, Not a File

Code is just **text files sitting in folders**. What turns a pile of files into a *project* is three things:

> **Important**
>
> This is why Claude Code, Codex, and Cursor ask you to open a **folder**, not a file. The root is the agent's unit of work: it reads the tree to learn the layout, follows imports across files, and runs your commands from there. "Open one file and fix it" is not how any of them work.

Splitting by Responsibility

Same amount of code. The difference is that on the right, each file answers **one kind of question** — so you can point at where a change belongs. So can an agent.

The Three Questions Every Request Answers

A request arrives. Three different *kinds* of decision have to happen, and each belongs somewhere different:

```
GET /api/discounts/42
  │
  ├─ router.py       "this URL maps to this function, and 42 is the customer id"
  │
  ├─ service.py      "customer 42 is strategic but 120 days overdue → 0%"
  │
  └─ repository.py   "SELECT * FROM customers WHERE id = 42"
```

> **Tip**
>
> Only the **service** layer knows the business. Swap Postgres for Snowflake and `service.py` never changes. Turn the web app into a nightly batch job and `service.py` *still* never changes.

Layers Alone Don’t Scale

The obvious next step is one file per layer. It works — for about six months:

Layers only

```
backend/
├── routers.py
├── services.py
└── repositories.py
```

Every endpoint in the product lives in one file. Every rule in the product lives in another. `services.py` becomes the new 800-line file.

Feature slices

```
backend/features/
├── customers/
└── discounts/
```

Each folder holds its **own** router, service, repository, and models — the same layers, scoped to one feature.

> **Important**

> You split by **technology**, but you change code by **feature**. “Add invoicing” should mean *add a folder* — not *edit three files that each contain nine unrelated things*.

A Real Project Layout

> **Note**
>
> This shape has a name: a **layered architecture, divided by feature**. The four layers — `router → service → repository → models` — repeat *inside* each feature folder, instead of being smeared across the whole app. You’ll also hear it called **feature-sliced** or **vertical slice** architecture.

The Entry Point Wires It Together

```
# backend/main.py
from fastapi import FastAPI

from core.config import settings
from core.database import connect
from features.customers.router import router as customers_router
from features.discounts.router import router as discounts_router

app = FastAPI()
connect(settings.DATABASE_URL)

app.include_router(customers_router, prefix="/api/customers")
app.include_router(discounts_router, prefix="/api/discounts")
```

> **Tip**
>
> Not one business rule in sight. `main.py` **decides nothing** — it wires. If you can read the entry point and list what the product does, the structure is working.

How Importing Works

Splitting files is only useful if the files can find each other:

```
from features.discounts.service import calculate_discount
#    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
#    folders, separated by dots          the function inside
#    features/discounts/service.py       service.py
```

- **Every dot is a folder step** — `features.discounts.service` *is* `features/discounts/service.py`
- The **file name is the module name** — no `.py` at the end
- Python resolves it **relative to where you ran the command**, so `python backend/main.py` and `cd backend && python main.py` are not the same thing
- Wrong folder → `ModuleNotFoundError: No module named 'features'`

> **Important**
>
> This is the single most common error after splitting files, and it is a **terminal** problem, not a Python problem. `pwd` first, then `ls`, then run.

Separation of Concerns

Now you can name the principle you've been watching:

> **Each file should have one reason to change.**

Change this…	…and only this has to change
Legal changes the overdue rule	`features/discounts/service.py`
You migrate Postgres → Snowflake	the `repository.py` files
The API response needs a new field	`features/customers/router.py`
Marketing restyles the page	`frontend/`
You add invoicing	one new folder under `features/`

> **Note**
>
> Read that column again: those are five different people, on five different timelines, with five different approval processes. **The architecture is an org chart that compiles.**

Reviewing What an AI Writes

You will not write most of this code. You will **accept** it. Run this checklist on the diff:

- Can you say what each file does in **one sentence**?
- Is any function longer than one screen?
- Is there a number in there that nobody can explain?
- Does a function name contain "and"?
- Did it duplicate logic that already existed somewhere else?
- Would a new teammate's WTF count be low?

> **Important**
>
> An agent will happily write 400 correct lines into one file. Correct today, unmaintainable next month. **Asking for structure is your job, not the model's.**

This is also why the next topic matters: small, well-separated files produce **small, readable diffs** — and a diff you can read is a change you can approve.

Git, GitHub & Version Control

The Problem: Tracking Changes

You're working on a project. You make changes, break something, and want to go back. Without version control:

- `project_final.py`
- `project_final_v2.py`
- `project_final_v2_FIXED.py`
- `project_final_v2_FIXED_for_real.py`

Git solves this — it tracks every change, lets you go back in time, and lets multiple people work on the same project.

What is Git?

Git is a version control system. It keeps a complete history of every change to every file.

- **commit** — a snapshot of your project at a point in time
- **branch** — a parallel version (experiment without breaking main)
- **merge** — combine changes from different branches
- **diff** — see exactly what changed

A commit is not “save.” It is a named, reviewable claim: **this coherent change should exist.** Git records intent over time.

What a Diff Looks Like

When you run `git diff`, you see exactly what changed — **red lines** were removed, **green lines** were added:

> **Tip**
>
> This is how code review works. Before merging, your team reads the diff to check what changed. You’ll see this view constantly in pull requests.

Git Branching — Visually

Branches make experimentation cheap. The best time to experiment is when you can compare, test, and undo without damaging the main work.

What is a Repository?

A **repository** (repo) is a project folder tracked by Git:

- Your code files
- The full history of changes (`.git/` folder)
- Configuration files

> **Tip**
>
> Every repo has a **main** branch — the “official” version. You create branches to work on features, then merge them back.

What is GitHub?

GitHub hosts Git repositories in the cloud. It adds:

- **Remote storage** — code is backed up online
- **Collaboration** — multiple people work on the same repo
- **Pull Requests** — propose changes, get code review before merging
- **Issues** — track bugs and feature requests
- **Actions** — automate testing and deployment (CI/CD)

> **Note**
>
> GitHub is to Git what Gmail is to email. Git is the technology, GitHub is the popular service. Alternatives: GitLab, Bitbucket.

Essential Git Commands

JS Reset ▶ Run

```
// Simulating git commands (these normally run in terminal)
const commands = [
  { cmd: "git init", desc: "Create a new repo in current folder" },
  { cmd: "git clone <url>", desc: "Download a repo from GitHub" },
  { cmd: "git status", desc: "See what files changed" },
  { cmd: "git add <file>", desc: "Stage changes for commit" },
  { cmd: "git commit -m 'msg'", desc: "Save a snapshot with description"
},
  { cmd: "git push", desc: "Upload commits to GitHub" },
  { cmd: "git pull", desc: "Download latest changes" },
  { cmd: "git branch <name>", desc: "Create a new branch" },
  { cmd: "git checkout <branch>", desc: "Switch to a branch" },
  { cmd: "git merge <branch>", desc: "Merge branch into current" },
];

console.log("Essential Git Commands:\n");
commands.forEach(({ cmd, desc }) => {
  console.log(`  ${cmd.padEnd(30)} → ${desc}`);
});
```

The Git Workflow — Visually

.

The Git Workflow

```
1. git pull                ← Get latest changes
2. git checkout -b feature   ← Create a branch
3. (write code)
4. git add .               ← Stage changes
5. git commit -m "Add feature" ← Save snapshot
6. git push                ← Upload to GitHub
7. Open a Pull Request       ← Ask for code review
8. Merge to main           ← Ship it!
```

> **Important**
>
> **Never commit secrets** (passwords, API keys, tokens) to Git. Once in history, it's very hard to remove. Use `.gitignore` to exclude sensitive files.

Commit Messages Are Documentation

Six months from now, someone (probably you) will ask: *"why does this line exist?"* The commit message is the answer.

Useless	Useful
`fix`	`Fix discount applying to overdue accounts`
`changes`	`Add strategic tier at 15% per 2026 pricing review`
`asdfasdf`	`Cap total discount at $50k (finance request)`

> **Tip**
>
> Format: **what changed and why**, in one line, present tense. AI agents write decent commit messages if you ask — but *you* should read them before they're immortalized in history.

Merge Conflicts — When Git Asks for Help

If two branches change the **same lines**, Git can't decide for you. It marks the conflict and waits:

```
<<<<<<< main
discount_rate = 0.10
=======
discount_rate = 0.15
>>>>>>> feature/new-pricing
```

You (a human, with context) pick the right version, delete the markers, and commit. Git handles everything else automatically — conflicts only happen when two people genuinely disagreed about the same line.

> **Note**
>
> Conflicts feel scary the first time. They're actually Git working as designed: **it refused to silently overwrite someone's work.** The horror stories all come from tools that *don't* do this.

Pull Requests — Where Business Meets Code

A **pull request (PR)** is a proposed change plus a conversation:

- The diff — exactly what would change
- A description — *why* it should change
- Review comments — teammates poke at the logic
- Checks — automated tests pass or fail, visibly
- A merge button — someone accountable clicks it

> **Tip**
>
> As a manager, PRs are your **audit trail**. Who changed the pricing logic? When? Who approved it? What did the tests say? It's all in the PR. This is also where you'll review AI-generated code — same process, same standards.

AI Coding Agents Need the Same Discipline

Everything above applies double when an AI writes the code:

- Give the agent a **concrete task**
- Inspect the **diff** — every changed line
- Run the relevant **check or test**
- Commit only what you **understand**

> **Warning**
>
> "The agent said it worked" is not a test.

Localhost: Running Software on Your Own Machine

Two Shapes of Software

Everything you ran today was a **script**: it starts, does its work, prints, and exits. That is only one of the two shapes software comes in.

- **A script** — runs once and finishes: `python3 report.py`
- **A server** — starts and *keeps running*, waiting to be asked for something: `python3 -m http.server`

  Every website you have ever used is the second kind. Somewhere right now, a program is running and waiting for you. When that program stops, the website is gone.

What `localhost` Actually Means

`localhost` is a name that always points back at the machine you are typing on. Its address is `127.0.0.1` — literally "this computer."

- Your browser asks for `http://localhost:8000`
- The request never touches the internet — no DNS, no wifi, no cable
- It goes from your browser to another program on **your own laptop**, and comes straight back

> **Note**
>
> This is why localhost still works on a plane with no wifi. You are the customer *and* the restaurant.

Ports: Apartment Numbers for One Address

One computer runs many programs at once. The **port** says which one you are talking to.

Address	Who usually lives there
`localhost:8000`	Python's built-in web server
`localhost:3000`	A Node / React / Next.js dev server
`localhost:5432`	A PostgreSQL database
`localhost:11434`	Ollama, running an LLM on your own machine

The address is the building; the port is the apartment. `Address already in use` means someone is already home — use a different port, or stop the program that's squatting there.

Your First Web Server — Zero Dependencies

Make a folder, put one file in it, run one command:

```
mkdir my-site
cd my-site
echo "<h1>Hello from my laptop</h1>" > index.html
python3 -m http.server 8000
```

Now open **`http://localhost:8000`** in your browser.
You installed nothing. Python ships with a web server. You are running one.

Why the Terminal “Freezes”

After that command your terminal stops taking input and prints something like:

```
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
127.0.0.1 - - [01/Sep/2026 16:04:11] "GET / HTTP/1.1" 200 -
```

- It is **not** frozen. It is *serving* — for a server, that is what “running” looks like.
- Each new line is one request your browser made. That is a **log**, and `200` means it worked.
- **Ctrl+C** stops it. Need to keep working? Open a second terminal tab.

What Happens on Each Request

Three moving parts: a browser that asks, a **process** that answers, and files it reads. Next class we replace that middle box with a real backend and a database — but the shape never changes.

The Site Is a Process, Not a File

Press **Ctrl+C** in the terminal, then reload the page.

> **This site can't be reached.** `localhost` refused to connect.

Your `index.html` is still sitting on disk, perfectly intact. The site is gone anyway.

A website is not a file. It is a program that has to be running, on a computer that has to be on.

> **Important**
>
> Close your laptop and your site dies with it. That one fact is the entire reason cloud hosting exists — and the entire reason anyone says "we can't deploy on Friday."

`localhost` Is Not the Internet

Send `http://localhost:8000` to the person sitting next to you.

- On their machine, `localhost` means **their** computer — not yours
- They get a connection error, or their own project if they happen to be running one
- The link is meaningless anywhere except your laptop

> **Note**
>
> Same room, same wifi, and it still doesn't work. Your laptop has no public address, and nothing on the internet knows how to find it. Giving it one is what **hosting** means — and it is the first thing we do in Class 2.

You Will Meet This Constantly with AI Agents

Ask an agent to build you a web app and it finishes with something like:

```
✓ Ready in 1.2s
  ▲ Local:   http://localhost:3000
```

- *"The agent said it's done but the page is blank"* — is the server still running? Look at the terminal.
- *"It worked yesterday"* — you rebooted. Restart the dev server.
- *"Port 3000 is already in use"* — you started it twice. Stop the old one.
- *"I sent my boss the link and it didn't load"* — you sent a `localhost` URL.

> **Tip**
>
> Understanding what "running locally" means turns four alarming bugs into four boring ones.

Setting Up Your Tools

What We're Installing

- **Node.js** — JavaScript runtime (needed for everything)
- **Git** — Version control (we covered this earlier)
- **Claude Code** — AI coding agent in your terminal
- **Vercel CLI** — Deploy websites with one command
- **Supabase CLI** — Database and auth in one command

> **Tip**
>
> Don't worry — we'll do this together. And once you have Claude Code, it can install the rest for you.

Installation

Mac

1. Install Homebrew

```
/bin/bash -c "$(curl -fsSL \
  https://raw.githubusercontent.com/\
  Homebrew/install/HEAD/install.sh)"
```

2. Install Node.js (includes npm & npx) and Git

```
brew install node git
```

3. Install the CLI tools via npm

```
npm install -g @anthropic-ai/claude-code
npm install -g @openai/codex
npm install -g vercel
npm install -g supabase
```

Windows

1. Download and install **Node.js** (LTS) — this gives you `npm` and `npx`
2. Download and install **Git**
3. Restart PowerShell, then run via npm:

```
npm install -g @anthropic-ai/claude-code
npm install -g @openai/codex
npm install -g vercel
npm install -g supabase
```

> **Note**

> You may need to **restart PowerShell** after installing Node.js for `npm` to work.

Or Just Ask Claude Code

Once you have Claude Code installed, you can ask it to do the rest:

```
claude "install vercel, supabase, and create-next-app CLIs for me"
```

> **Tip**
>
> This is the whole point — Claude Code can run terminal commands for you. You don't need to memorize any of this.

AI Coding Agents

Claude Code & OpenAI Codex

These are **AI agents that run in your terminal** — not chatbots in a browser.

	Claude Code	OpenAI Codex
By	Anthropic	OpenAI
Launch	`claude`	`codex`
YOLO	`--dangerously-skip-permissions`	`--yolo`
Models	Sonnet, Opus, Haiku	GPT-5.4, 5.4 mini, 5.4 nano

Both can read your codebase, write files, run commands, and iterate autonomously. Use whichever you prefer — the workflow is the same.

Why the CLI Is the Most Powerful Version

- **Full filesystem access** — it can read any file in your project, not just what you paste
- **Runs commands** — it can install packages, run builds, execute tests
- **Iterates autonomously** — run tests → see failure → fix → repeat
- **Works with any project** — not locked to one editor or platform
- **Multiple model versions** — pick speed vs depth depending on the task

> **Important**
>
> The chat UI (claude.ai, ChatGPT) is great for questions. But for **building software**, the CLI is in a different league.

YOLO Mode

Skip all permission prompts — let the AI just go:

```
# Claude Code
claude --dangerously-skip-permissions

# OpenAI Codex
codex --yolo
```

> **Warning**
>
> **YOLO mode** means the AI will create files, install packages, and run commands **without asking you first**.
>
> Use it for greenfield projects where you trust it to figure it out. Don't use it on production code you care about.

Think of it like autopilot. Great for takeoff on an empty runway. Maybe not for landing in a thunderstorm.

Plan Mode

Start with a plan before letting Claude execute:

1. Type `/plan` in Claude Code to enter plan mode
2. Describe what you want to build
3. Claude creates a detailed plan — **review it**
4. Exit plan mode and let Claude execute

> **Tip**
>
> **Best workflow:** Plan first (so you understand what's about to happen), then run in YOLO mode to execute the plan without interruptions.

When NOT to Reach for the Agent

AI coding agents are powerful. They are not always the right tool:

- **A one-line change you understand** — just make it. Faster than explaining it.
- **A decision, not a task** — "should we use Postgres or MongoDB?" needs *your* context (team, budget, risk), not autocomplete.
- **Anything touching production data** — slow down, get a human review, use a staging copy.
- **When you can't evaluate the output** — if you can't tell good from bad, the agent's confidence is worthless to you. Learn just enough first.

> **Note**
>
> The pattern for this whole course: **automate the work, never the accountability.**

Plain-Language Reasoning Improves AI Output

Before asking an agent to code, describe the **inputs, rules, exceptions, and success cases** in ordinary language. Compare:

- *"make a discount thing"*
- *"Write a function that takes revenue and tier. Gold gets 10%, strategic gets 15%, anyone with an overdue invoice gets 0% unless an exception is approved. Here are three examples with expected outputs."*

> **Tip**
>
> Clear thinking arrives before clear code. The second prompt gets a better result from any model — because the *thinking* was already done.

Explain the Program Before You Run It

If you cannot predict the output of a small example, slow down and trace the values line by line.
That habit catches more errors than memorizing syntax — and it's exactly the skill you need to review AI-generated code you didn't write.

Before You Accept an Agent's Diff

A concrete checklist — the same one professional reviewers use:

- **Does it match the brief?** The agent solved *a* problem. Is it *your* problem?
- **Any files you didn't expect?** New config files, deleted tests, changed dependencies.
- **Any secrets?** API keys or passwords pasted into code instead of environment variables.
- **Do the tests still pass?** Run them. Don't take the agent's word for it.
- **Can you explain the change out loud?** If not, ask the agent to explain it — then verify the explanation against the code.

> **Important**
>
> This checklist takes two minutes. Skipping it is how vibecoded prototypes end up in the "Bugs That Cost Real Money" table.

Exercise: Ship the Discount Engine

The Brief

Build the `discount-engine` we've been sketching all class — for real, with an agent.

- It applies today's rules: tier rates, an overdue cutoff, an approval exception, a cap
- It runs on **your machine**, and you can open it in a browser
- It is structured the way we just covered — not one 800-line file

You are not writing this alone. **Claude Code writes it. You direct it, review it, and own it.**

The Prompt

Start here. Change anything you like — it's your project:

```
Build a small Python discount engine and serve it on localhost.

Business rules:
  - standard 5%, gold 10%, strategic 15%
  - any invoice more than 90 days overdue gets 0%,
    unless an exception is approved
  - no discount is ever larger than $25,000

Separate the business rules from the data and from the web layer.
Write unit tests for the rules. Include a README.

Use plan mode first — show me the plan before you write any code.
```

> **Tip**
>
> Read the plan **before** you approve it. If it proposes one big `main.py`, say so and ask for the split. **That correction is the exercise.**

Definition of Done

Six things. Bring them to Class 2:

- **README** — what it is and how to run it
- **Sensible structure** — you can say in one sentence what each file does
- **Running on localhost** — you opened it in a browser and watched it work
- **Unit tests** for the rules, and they pass
- **Plan mode used** — you read the plan and approved it deliberately
- **Pushed to GitHub** — with commit messages a human can read

> **Important**
>
> If you can't explain what a file does, you're not done — you're just holding code somebody else wrote. Ask the agent to explain it, or ask it to simplify until you can.

Vocabulary You Can Now Use in Meetings

Words that meant nothing to you two hours ago:

Term	What you can now say
variable / state	“What was the state when it failed?”
function	“Is that logic in one function or copy-pasted everywhere?”
refactor	“Same behavior, better structure — no new features in this one.”
module / entry point	“Which file does that live in, and what calls it?”
separation of concerns	“Why does the pricing change touch the report code?”
technical debt	“We shipped it fast — what did we borrow to do that?”
logic error	“It runs, but has anyone tested the edge cases?”
dependency	“What libraries does this pull in, and who maintains them?”
repo / branch / PR	“Open a PR — I want to see the diff before we ship.”
diff	“Walk me through the diff.”

Term	What you can now say
regression	“Did we add a test so this can’t break again?”
YOLO mode	“Please tell me you didn’t run that in YOLO mode on prod.”

> **Tip**
>
> You don’t need to write production code to be dangerous in a technical meeting. You need the vocabulary, the mental model, and the confidence to ask “show me.”

Next Class

Your discount report is a real website — running at an address only you can reach, on a machine that has to stay open, with no way to let anyone in.

A **product** is the system that fixes all of that: hosting, a public URL, a backend, a database, authentication, tests, and deployment.

Next: Class 2 — Software Systems →

Speaker notes